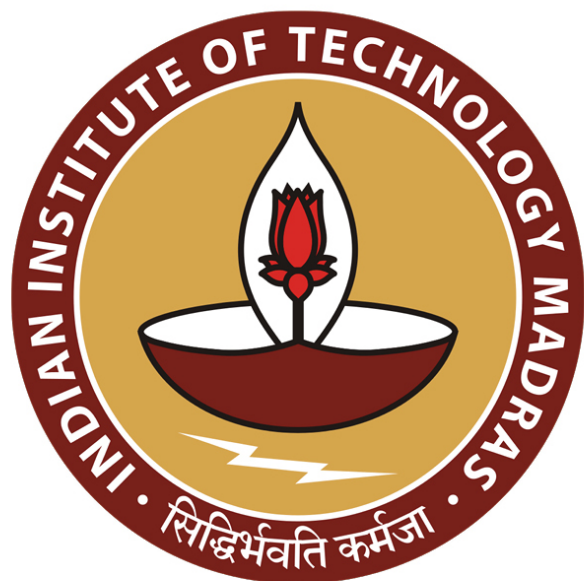


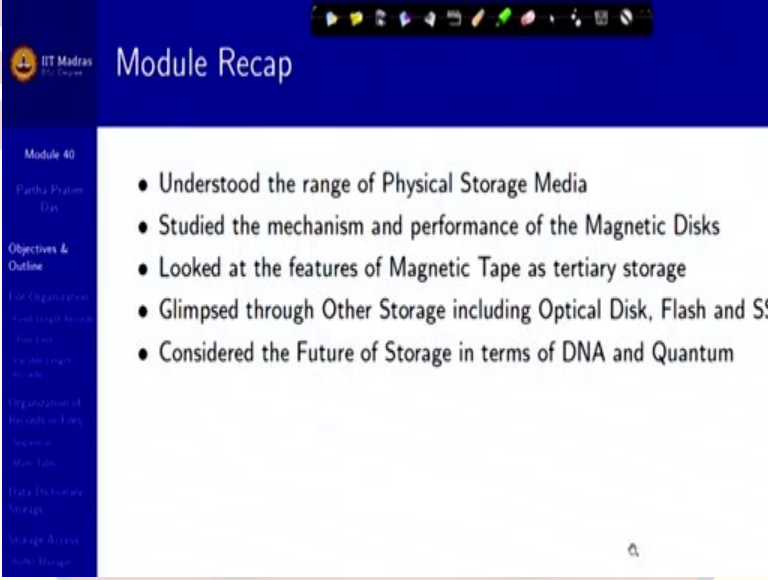


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Storage and File Structure/2: File Structure

Welcome to module 40 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 0:28)



Module Recap

- Understood the range of Physical Storage Media
- Studied the mechanism and performance of the Magnetic Disks
- Looked at the features of Magnetic Tape as tertiary storage
- Glimpsed through Other Storage including Optical Disk, Flash and SS
- Considered the Future of Storage in terms of DNA and Quantum

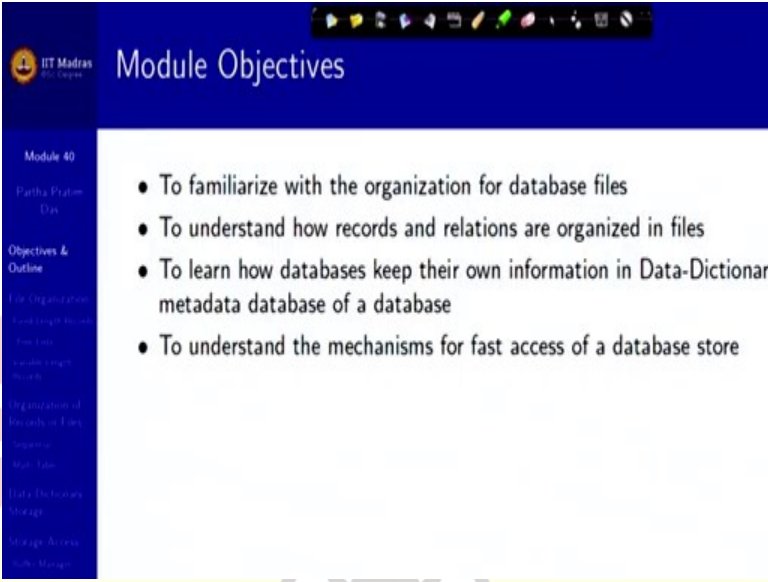
Module 40
Partha Pratim Das

Objectives & Outline

- File Organization
- File System
- File Access Methods
- Organization of Records in Files
- Indexing
- Index Files
- Data Dictionary
- Storage Access
- File Management

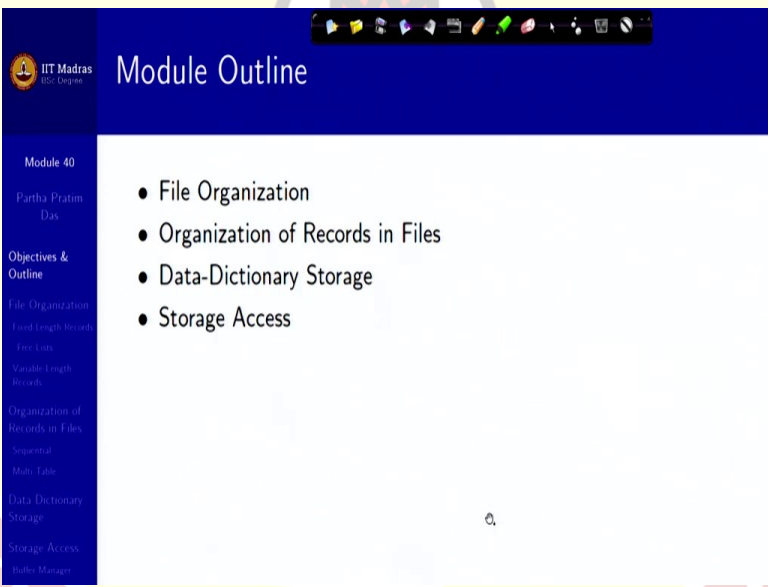
In the last module we took a quick look at different physical storage media options, particularly the secondary storage in terms of magnetic disk which is going to be our mainstay of operations and also the tertiary storages and other storage media as well as future.

(Refer Slide Time: 0:51)



Module Objectives

- To familiarize with the organization for database files
- To understand how records and relations are organized in files
- To learn how databases keep their own information in Data-Dictionary metadata database of a database
- To understand the mechanisms for fast access of a database store



Module Outline

- File Organization
- Organization of Records in Files
- Data-Dictionary Storage
- Storage Access

Now, we are going to look at how to organize our database files in this storage media. So, I would rather, I should rather say that I will start with the logical structures of the physical layer, how database files should be organized, how records in that should be put, what will be the data dictionary and what would be the different fast access mechanisms.

(Refer Slide Time: 1:33)

IIT Madras
File Organization

Module 40
Partha Pratim Das
Object Oriented & OO DBMS
File Organization
Fixed Length Records
File Size
Variable Length Records
Organization of Records in Files
Sequential
Hash Files
Data Dictionary
Storage
Storage Access
Indexing

- A database is
 - A collection of *files*. A file is
 - ▷ A sequence of *records*. A record is
 - A sequence of *fields*
- One approach:
 - assume record size is fixed
 - each file has records of one particular type only
 - different files are used for different relations
 - This case is easiest to implement; will consider variable length rec
- A database file is partitioned into fixed-length storage units called **blc**
 - Blocks are units of both storage allocation and data transfer

So, for file organization consider that a database has a hierarchical containment. A database is a collection of files. A file is a sequence of records. A record is a sequence of fields. So, this is the basic mental model we start with, this is how we will organize. So, one approach that we can take just to get started in simple terms that we can assume that the record have fixed size.

We can also assume that records are of only one type. We assume that different files are used for different relations, so we are just, so every file is for a different relation, every record is of one type only and the size of the record is fixed. Of course this is not in general will be true but this is just to get started so that we can have a easy to implement formulation, then we will generalize to variable length records and so on.

(Refer Slide Time: 2:50)

File Organization

- A database is
 - A collection of *files*. A file is
 - ▷ A sequence of *records*. A record is
 - A sequence of *fields*
- One approach:
 - assume record size is fixed
 - each file has records of one particular type only
 - different files are used for different relations
 - This case is easiest to implement; will consider variable length rec
- A database file is partitioned into fixed-length storage units called **blc**
 - Blocks are units of both storage allocation and data transfer

At this point I would like to remind you that the database file is partitioned into fixed length storage units, we talked about this in terms of the, in terms of magnetic disk we mentioned that while sector is a basic unit to write blocks a couple of sectors could build a block, which is the fixed length unit which is in terms of which we do the storage allocation as well as data transfer.

(Refer Slide Time: 3:20)

Fixed-Length Records

- Simple approach:
 - Store record i starting from byte $n * (i - 1)$, where n is the size of
 - Record access is simple but records may cross blocks
 - ▷ Modification: do not allow records to cross block boundaries
- Deletion of record i : Alternatives:
 - move records $i + 1, \dots, n$ to $i, \dots, n - 1$
 - move record n to i
 - do not move records, but link

record	id	name	dept
record 0	10101	Srinivasan	Comp.
record 1	12121	Wu	Finance
record 2	15151	Mozart	Music
record 3	22222	Einstein	Physics
record 4	32343	El Said	History
record 5	33456	Gold	Physics
record 6	45565	Katz	Comp.
record 7	58583	Califieri	History
record 8	76543	Singh	Finance

So, a simple approach could be just to follow a linear array like structure, we say, we keep the, store the records because each one of them is of same size, so we just keep them one after the other and if we do that, then if n is the size of each record, then the i th record will be available

starting from this address, so you know the offset just like that array and record access will get simple but we have to make sure that records do not cross the block boundary. So, if it crosses a block boundary then some more computation will have to be done.

Now, in terms of deletions what alternatives do we have? If we delete some record, suppose we are deleting this record, then what are the options? One is that you move like the array you move the rest of the records up, all of these records up is one option. Other could be that instead of moving this up you take the last record and move it here, so you can move all of them up just move the last record.

Or a third could be that you do not move anything but just mark that this particular record is no more in use and it is that space is free and chain them up as a free list. So, these are the three basic strategies we can have.

(Refer Slide Time: 4:50)


IIT Madras
Indian Institute of Technology Madras

Deleting Record 3 with Compaction

Module 40

Partha Pratim Das

**Object-Oriented
Database**

**File Organization
Fixed Length Records**

**Free Space
Management**

**Organization of
Records on Disk**

**Indexing
Main File**

**Data Dictionary
Usage**

**Storage Access
Buffer Manager**

Before deletion

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

After deletion & Co

record 0	10101	Srinivasan	C
record 1	12121	Wu	F
record 2	15151	Mozart	N
record 3	32343	El Said	F
record 4	33456	Gold	F
record 5	45565	Katz	C
record 6	58583	Califieri	F
record 7	76543	Singh	F
record 8	76766	Crick	E
record 9	83821	Brandt	C
record 10	98345	Kim	E
record 11			

So, if we do compaction that is if we remove the record 3 and then move the records, remaining records up this is what we get, you can understand this is going to be expensive.

(Refer Slide Time: 5:10)

record	ID	Name	Subject	Value
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

record	ID	Name	Subject	Value
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Now, if we do a movement, so what I, we are doing is we delete this and we move record 11 here, so record 11 has come here, this ends in record 10, so that is another option which may be good in many places.

(Refer Slide Time: 5:26)

- Store the address of the first deleted record in the file header
- Use this first record to store the address of the second deleted record
- Consider these stored addresses as pointers since they point to the
- More space efficient representation: reuse space for normal attributes store pointers (No pointers stored in in-use records)

header	ID	Name	Subject	Value
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	15151	Mozart	Music	40000
record 2	22222	Einstein	Physics	95000
record 3	33456	Gold	Physics	87000
record 4	58583	Califieri	History	62000
record 5	33456	Gold	Physics	87000
record 6	58583	Califieri	History	62000
record 7	58583	Califieri	History	62000

More common is using a free list, store the address of the first deleted record in the file header, in the header of the file you write the address of the record, first record that has been deleted. Use this record to store the address of the second record that is deleted, so that is what, so these are the free spaces so you are just chaining them up as if as a pointer.

Actually, you do not need to have this separate pointer what you can do because you are chaining only the free spaces, they are not having the normal attribute value. So, you can use that space itself, a part of that space itself as to mark that it is free and also put the pointer to the next free record.

So, that way you do not put any space over it but you can get, when you have to insert you can check if you have a free space for a record and you can put your new record there. So, these are the three ways typically a fixed length record file can be managed.

(Refer Slide Time: 6:46)

Variable-Length Records

- Variable-length records arise in database systems in several ways:
 - Storage of multiple record types in a file ✓
 - Record types that allow variable lengths for one or more fields such as (**varchar**)
 - Record types that allow repeating fields (used in some older data)
- Attributes are stored in order ✓
- Variable length attributes represented by fixed size (offset, length), w stored after all fixed length attributes
- Null values represented by null-value bitmap

Null bitmap (stored in 1 b)
0000

But in general, the records are variable length, because you may store multiple record types in the file. The record types allow fields, one or more fields which are themselves variable in size like varchar, you do not know how long it will be, some older record types allow repeating fields and so on.

So, what is done is simple the attributes are stored in order, again in the order in which you have defined. If you have a variable length attribute you store it as a fixed size like in here. So, what you say, you say you do not store the value here, rather you store an offset of where the value can be found later on. So, you say that it is 21, 5; so this value can be found here and its length will be 5.

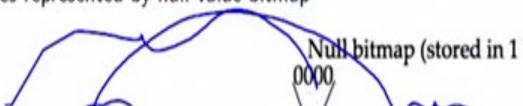
(Refer Slide Time: 7:50)

**IIT Madras**
Vellore, India

Variable-Length Records

Module 40
Partha Pratim Das
Objectives & Outline
File Organization
Fixed Length Records
File Cues
Variable Length Records
Organization of Records in Files
Sequences
Main Table
Data Dictionary
Storage
Storage Access
Index Manager

- Variable-length records arise in database systems in several ways:
 - Storage of multiple record types in a file
 - Record types that allow variable lengths for one or more fields such as (**varchar**)
 - Record types that allow repeating fields (used in some older data)
- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), w stored after all fixed length attributes
- Null values represented by null-value bitmap



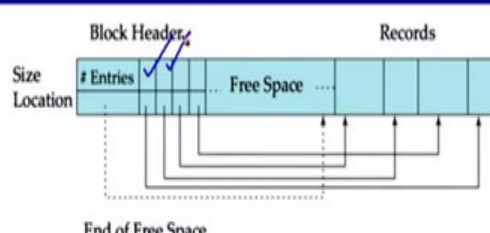
Similarly, the next value can be found at 26 which is Srinivasan, and its length is going to be 10. And you have these variable length fields after all fixed length attributes. So, kind of you have a fixed part, you do not have to really keep on searching as to where does this variable stuff end and what does my next attribute start you do not have to do any of that, this fixed length part gives you everything, you know this order, you know what are each of the size so you can take any attribute directly. And if it is a variable length you need to just traverse this pointer and get the actual value. And null values are represented by null value bitmap, as we have seen here.

(Refer Slide Time: 8:47)

**IIT Madras**
Vellore, India

Variable-Length Records (2)

Module 40
Partha Pratim Das
Objectives & Outline
File Organization
Fixed Length Records
File Cues
Variable Length Records
Organization of Records in Files
Sequences
Main Table
Data Dictionary
Storage
Storage Access
Index Manager



- Slotted Page** header contains:
 - number of record entries ✓
 - end of free space in the block ✓
 - location and size of each record ✓
- Records can be moved around within a page to keep them contiguous

Now, often what we do is we basically, because a block will have multiple records. So, it will be good to have what is called a slotted page, where the header contains a lot of information. What are the information? It says, what are the number of record entries, how many records are there. Where is the end of free space in the block and the location and size of each record? So, it is like having a header information, a dictionary information for the records that you have.

So, then the records can be moved around in the page to keep them contiguous. Because nobody is referring to them directly. If somebody has to refer, he has to refer to the record in the entry in the header, not to the record directly. And of course, if you move the record around you have to update the headers but that is internal to the block.

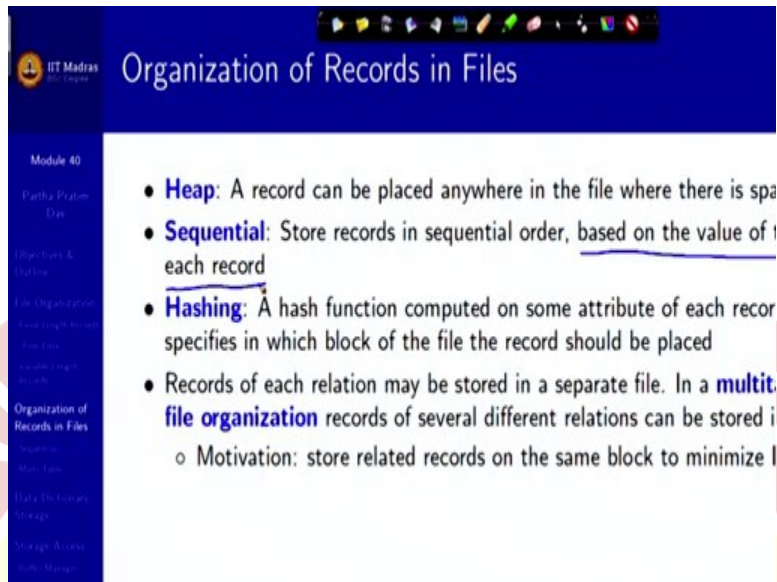
So, that way it becomes very efficient, externally you cannot see this rearrangement that you need because of insertion of new records, deletion of some records or change of value for a variable length field and so on, so this is slotted page with header is a usual structure for.

(Refer Slide Time: 10:24)



So, that was about the basic organization of records in terms of the file.

(Refer Slide Time: 10:33)



Organization of Records in Files

- **Heap:** A record can be placed anywhere in the file where there is space.
- **Sequential:** Store records in sequential order, based on the value of the search key of each record.
- **Hashing:** A hash function computed on some attribute of each record specifies in which block of the file the record should be placed.
- Records of each relation may be stored in a separate file. In a **multi-file organization** records of several different relations can be stored in the same file.
 - Motivation: store related records on the same block to minimize I/O.

Now, we will now see how they will actually be stored. So, we have seen the logical view, now let us see how will they actually physically occur. So, it could be a heap, a heap is I mean, you just keep things anywhere and everywhere, we talk about heap in the computer memory regularly for dynamic allocation. It could be sequential where records are stored in sequential order based on the value of the search key of each record, that is the tricky part, based on the value of the search key of each record.

Why are you doing this? Because if it is done in a sequential order based on that value, it can make you do something like a binary search more easily and that is why you want the search key, the key basically. It could be based on hashing where you use a separate function using some of the attributes of the record and we will talk more about this. Or it could be multi table also. For example, you might want to have multiple relations of your design in the same table, it is called multi table clustering organization, it might minimize your I/O.

(Refer Slide Time: 12:04)


IIT Madras
Indian Institute of Technology Madras



Sequential File Organization

Module 40

Partha Pratim Das

Operating System

File Organization

Sequential Access

File Data

Sequential Access

Records

Organization of

Records in Files

Sequential

Access

Data Dictionary

Storage

Storage & Access

File Manager

- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a search-key

10101	Srinivasan	Comp. Sci.	65000	✓
12121	Wu	Finance	90000	✓
15151	Mozart	Music	40000	✓
22222	Einstein	Physics	95000	✗
32343	El Said	History	60000	✗
33456	Gold	Physics	87000	✗
45565	Katz	Comp. Sci.	75000	✗
58583	Califieri	History	62000	✗
76543	Singh	Finance	80000	✗
76766	Crick	Biology	72000	✗

So, let us look at some of these. So, this is a sequential organization. So, what you do is it is suitable for applications that need sequential processing of the entire file, so you keep them one after the other and these links basically tell you the order in terms of the search key.

(Refer Slide Time: 12:28)



IIT Madras
© 2019

Sequential File Organization (2)



Module 40

Partha Pratim Das

Operating System

File Organization

Sequential File Organization

Indexing

Hashing

Journaling

Backup and Recovery

Security

Performance

Storage Management

- Deletion: Use pointer chains ✓
- Insertion: Locate the position where the record is to be inserted
 - if there is free space insert there ✓
 - if no free space, insert the record in an overflow block
 - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order ✓

10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
15151	Mozart	Music
22222	Einstein	Physics
32343	El Said	History
33456	Gold	Physics
45565	Katz	Comp. Sci.
58583	Califieri	History
76543	Singh	Finance
76766	Crick	Biology
83821	Brandt	Comp. Sci.
98345	Kim	Elec. Eng.

32222	Verdi	Music
-------	-------	-------

So, what will happen if you have to delete all that you will need is to change this pointer link, you will just skip that, put it to the next and whatever got freed you will put it to the free list. So, pointer management deletion is done.

For doing the insertion you will have to check if there is a free space from the free list, if there is free space you put there, if it is not then the block has run out of space in that case you may have to allocate another block which is called the overflow block. And whatever you do you have to now it is like a insertion in the link list, so you will have to basically insert this element using the pointer.

So, in this way they may have too many of pointers going this way, that way, so periodically you may need to do a reorganization and clean things up, just make them more sequential that is, at a leisure time of the disk you could do this reorganization. So, that is a typical sequential organization.

(Refer Slide Time: 13:42)

Module 40
Partha Pratim Das
Objectives & Outline
File Organization
Contiguous Records
Indexed Sequential
Organization of Records in Files
Inverted
Multi-Table
Data Dictionary
Storage
Storage Access
Indexing

Multitable Clustering File Organization

Store several relations in one file using a **multitable clustering** file organization.

department		dept_name	building	budget
		Comp. Sci.	Taylor	100000
		Physics	Watson	70000

instructor		ID	name	dept_name	salary
		10101	Srinivasan	Comp. Sci.	65000
		33456	Gold	Physics	87000
		45565	Katz	Comp. Sci.	75000
		83821	Brandt	Comp. Sci.	92000

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000

We are, I am not going to talk about the hashing organization because as I said before we will dedicate a separate module on it. Another is multi-table clustering. So, just let us start with an example. So, we have seen this relation number of times the department, which has department name, building, budget. And which has also another table instructor, ID, name, department name, salary and so on. So, normally what you will expect? You will expect to keep them separate.

Now, I do something interesting. What I do is I will group the records of instructor who are in the same department together. If I do that, I do not need to remember this, I do not need to write this because they are all from Computer Science. So, I will write a row for this department as I

write here and then I will put the three who are from that department. Remind you from this the department name from the instructor the department name I do not need to keep.

Then I put the next department, this is a department record and I put the instructor who is working in that department, it is very different from the way you have logically designed. So, what you are getting is if you get a depart, because what is the rational behind this? The rational is that there is a many to one relationship, many instructor to one department.

So, there is a scope of lot of grouping and since there is a many to one relationship, your expectation is there will be lot of queries which will be done for a specific department, if that assumption is not correct then this organization is not good, but often that will happen if you assume, if that happens then what is.

Then all that you need is to come to the department and then rest of it is available together, it becomes lot more efficient in terms of access and processing. So, multi cluster, I mean of course there are issues because here the design is taken up in such a way that this has three attributes, this has three attributes kind of it matches and so on.

But there are different types like on this there is a varchar Computer Science whereas this may have been a number ID and so on but we will have to take care of those variables and variable types and variable size of records between the two relations but conceptually this is what can give you a lot of benefits and often that is used that way.

(Refer Slide Time: 16:47)

IIT Madras
Module 40
Partha Pratim Das
Objectives & Outline
File Organization
Fixed Length Records
Free Space
Variable Length Records
Organization of Records in Files
Sequential
Multi Table
Data Dictionary
Storage
Storage Access
Indexing

Multitable Clustering File Organization (2)

- good for queries involving department $\bowtie$ instructor, and for queries in department and its instructors
- bad for queries involving only department
- results in variable size records
- Can add pointer chains to link records of a particular relation

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	

IIT Madras
Module 40
Partha Pratim Das
Objectives & Outline
File Organization
Fixed Length Records
Free Space
Variable Length Records
Organization of Records in Files
Sequential
Multi Table
Data Dictionary
Storage
Storage Access
Indexing

Multitable Clustering File Organization (2)

- good for queries involving department $\bowtie$ instructor, and for queries in department and its instructors
- bad for queries involving only department ✓
- results in variable size records ✓
- Can add pointer chains to link records of a particular relation ✓

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	

So, it is good for involving queries which has department cross instructor, because department join instructor, because you have to join department with instructor, so you have to go by the department, take all instructors which is already done here, already done here. It is good for queries which have a single department and its instructors.

So, you can see that these we are putting together because they all work for that department and then we are chaining the departments because it is done department wise. Again this is possible because of the many to one property of the relationship. Now, it is certainly bad for queries which just have departments because now they are not sequential they have to go on the chain,

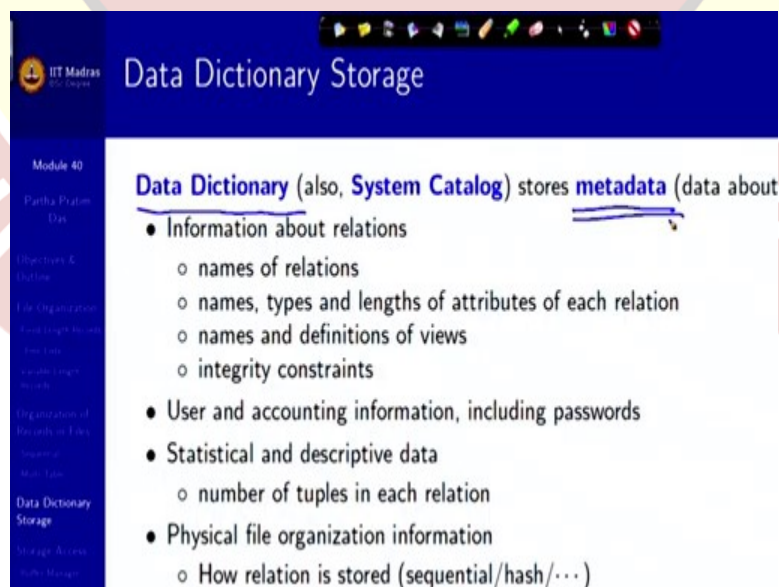
this will obviously as I said will result in variable size records. Of course, we can add pointer chain to link records of a particular relation, there are different designs which do it in this way.

(Refer Slide Time: 17:54)



So, we saw the sequential, heap is does not have an organization so I skipped, we saw the sequential as well as we saw the multi-table clustering of multiple relations and we will talk about hashing in the later module.

(Refer Slide Time: 18:10)



The next that we have to look at is I mentioned that there is the database itself has lot of information to manage, what are the relations, what are the different attributes of every relation,

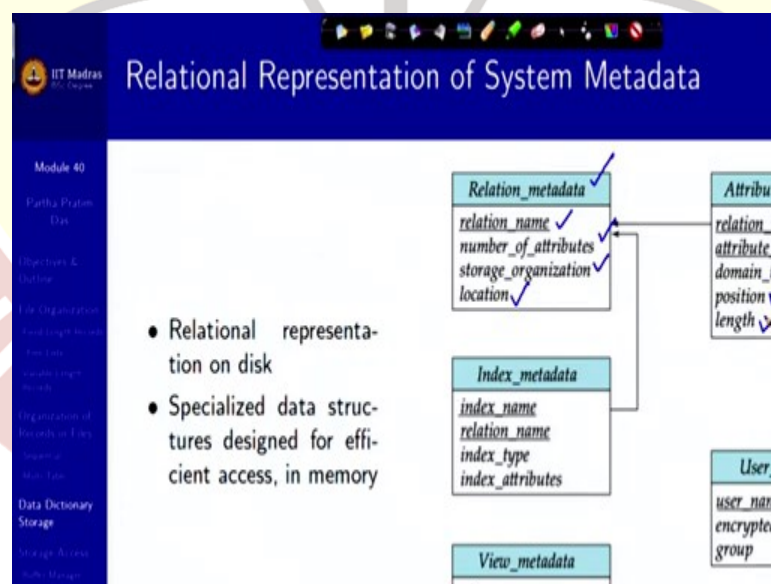
what is the types of these attributes, what are the different views, what are the integrity constraints, who are the users, what is the kind of physical structure and strategies you are following and so on so forth.

So, database recursively uses itself by having a database called the data dictionary, it is often called the system catalog also, which stores the metadata, metadata is data about data because everything is a data and you are keeping data about that, so you are keeping data about relations, about attributes, about users and so on.

You keep that in a separate database design and that is the data dictionary. So, data dictionary has to be done very efficiently because every time you have to do something, you have to go through the data dictionary to know where that relation is or what is that type this relation is satisfying and so on and so forth.

So, it may be that you want to make sure that the data dictionary which is not expected to be very large can always stay in the main memory and does not have to go back and forth with the disk which will drastically reduce the performance.

(Refer Slide Time: 19:42)



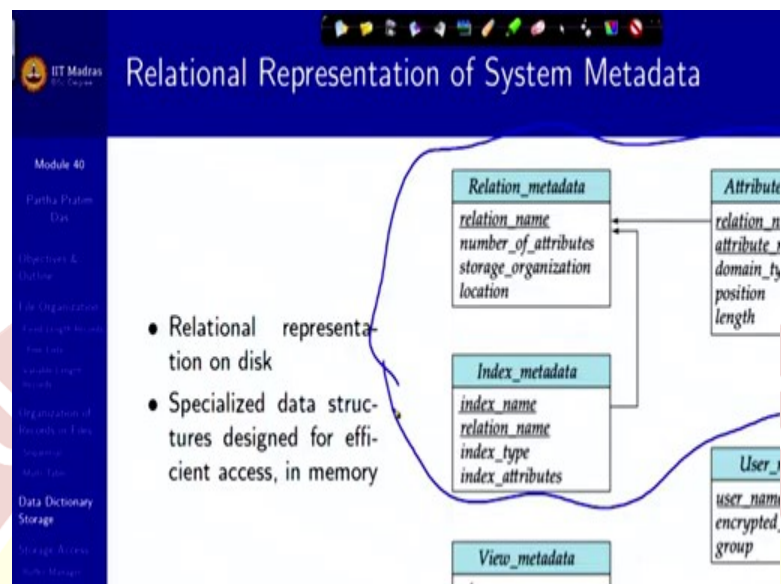
So, if we just look at some typical examples, so there is a, there will be a table like relational metadata which has relation name, number of attributes, what kind of storage organization you have opted for in this relation, where is the location where it can be found, and there is attributes

Attribute name, its domain type, its position because I said they will have to be in the specific order to be able to quickly so that we can quickly locate it by offset, so the position is that kind of the offset and the length position is that I should say the ordinal number where is the first attribute, second attribute, seventh attribute and so on, so that I can add all of those and get that number of wire to find it and the length.

Relational Representation of System Metadata

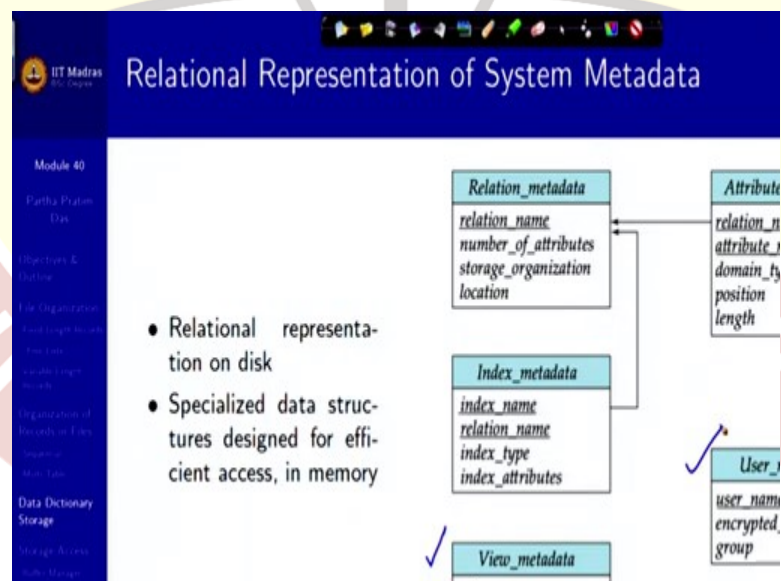
We may have indexes on the relation, we have not yet talked about index, index is the structure by which we can search a relation very efficiently, we will discuss about index in the next week. But index 1 or more, indexes may be 0 or more, indexes may be created on a relational schema relation. So, it is again another many to one relationship as you can see here. So, index has a name, it has a relation for which it is index, index has different types, it has indexing attributes and so on.

(Refer Slide Time: 21:23)



So, if you just look at the even these three without any specific constraints or anything it tells you a lot about how you have to manage your databases own data, about the relations, indexes, attributes and so on, there would obviously be lot more.

(Refer Slide Time: 21:40)



Then there are other tables like all the different views that you have defined, the user metadata, the user names, the encrypted password, group, permissions, authenticity, rights and all of that would be there. So, all those that go into the data dictionary.

(Refer Slide Time: 21:57)



Storage Access

Module 40

Partha Pratim Das

Objectives & Outline

File Organization

Fixed Length Records

Free Space

Variable Length Records

Organization of Records in Files

Indexing

Hash Tables

Data Dictionary

Storage

Storage Access

Buffer Manager

- A database file is partitioned into fixed-length storage units called blocks
 - Blocks are units of both storage allocation and data transfer
- Database system seeks to minimize the number of block transfers between memory
 - We can reduce the number of disk accesses by keeping as many blocks in main memory
- **Buffer**: portion of main memory available to store copies of disk blocks
- **Buffer Manager**: subsystem responsible for allocating buffer space in memory

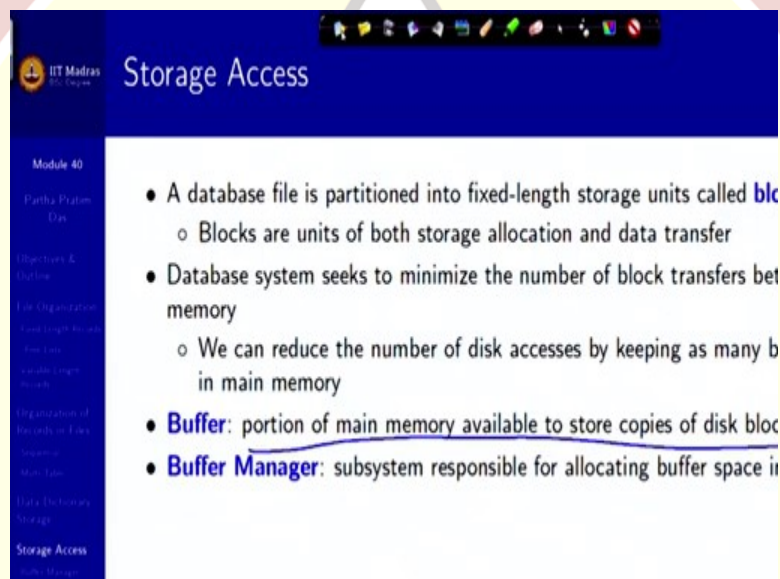
So, next we talk about the access of the storage. As we have said that the data is in secondary store which is slow on the disk and so on but huge. And the operations are data structures in the main memory which has to be small, cannot be very large but very fast to operate on our actual operations will happen in the memory data.

So, which means the anything that I need to do I have to bring the data from the secondary store, the disk to the memory, operate and put it back. Now, if I do it for every data item every time, then I will be spending all my time to just to read the data, write the data back, read the data, write the data back.

First of all, reading the data from the disk itself is a much slower process than memory access, one. Second, there is a lot of network cost involved in transferring the data from the secondary disk, from the magnetic disk to the main memory. So, we have to minimize the requirement of such read and write transfers, so that is a basic issue.

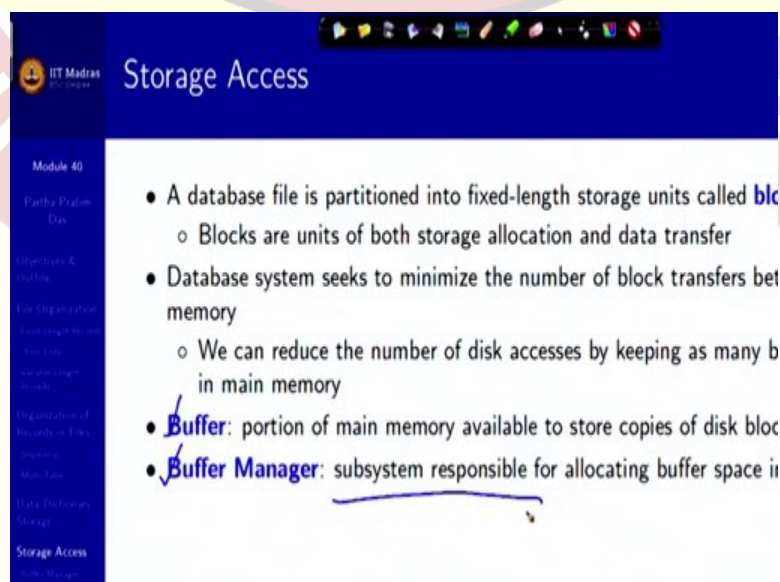
So, what we do is we have divided the storage in terms of blocks I mentioned earlier so these are kind of units in which you will do the data transfer. And what we try to minimize is the number of block transfers between the disk and the memory. So, how we can minimize this? If we can keep more and more blocks in the main memory.

(Refer Slide Time: 23:55)



Storage Access

- A database file is partitioned into fixed-length storage units called **blc**
 - Blocks are units of both storage allocation and data transfer
- Database system seeks to minimize the number of block transfers bet memory
 - We can reduce the number of disk accesses by keeping as many b in main memory
- **Buffer**: portion of main memory available to store copies of disk bloc
- **Buffer Manager**: subsystem responsible for allocating buffer space in



Storage Access

- A database file is partitioned into fixed-length storage units called **blc**
 - Blocks are units of both storage allocation and data transfer
- Database system seeks to minimize the number of block transfers bet memory
 - We can reduce the number of disk accesses by keeping as many b in main memory
- **Buffer**: portion of main memory available to store copies of disk bloc
- **Buffer Manager**: subsystem responsible for allocating buffer space in

So, for that we allocate a area of the main memory to store copies of test blocks and that is called buffer. Once we do that then the main issue turns out to be how we manage this buffer, buffer management becomes a big question. So, we have a buffer manager, a subsystem responsible for allocating the buffer space in main memory and managing it.

(Refer Slide Time: 24:29)

Buffer Manager

Module 40

- Programs call on the buffer manager when they need a block from disk
 - If the block is already in the buffer, buffer manager returns the address in main memory
 - If the block is not in the buffer, the buffer manager
 - ▷ Allocates space in the buffer for the block
 - Replacing (throwing out) some other block, if required, to the new block
 - Replaced block written back to disk only if it was modified recent time that it was written to / fetched from the disk
 - ▷ Reads the block from the disk to the buffer, and returns the address in main memory to requester

So, let us look at some of the basic issues that are involved, what will happen with a buffer manager. So, the program calls on the buffer manager saying that I need a block from the disk, send a block from the disk, discuss a block as ID and all that he says that. So, what are the possibilities? One is the block may already be there in the buffer, it was brought earlier by the program, so it may be already there in the buffer.

Then the buffer manager will simply return that address to the program that this is where your buffer is, just one block. The other possibility is the block is not there in the buffer, if it is not there in the buffer it has to be brought from the disk to the memory, if it has to be brought from the disk to the memory, then I need space in the buffer to put it.

Now, if the buffer already has space I will be able to bring the block and keep it there. If not, then we have to replace or throw out some existing block because it is a limited space and then by that process makes space. Now, whatever you throw out already have updated data, so whatever you are replacing will may need to be written back to the disk if it was modified.

From the last time it was written back or last time it was fetched. Now, if it was not modified after that you do not need to write back, you can just use that space, the disk already has a identical copy, but if it was modified you have to write back. So, with this process the allocation gets completed.

(Refer Slide Time: 26:24)

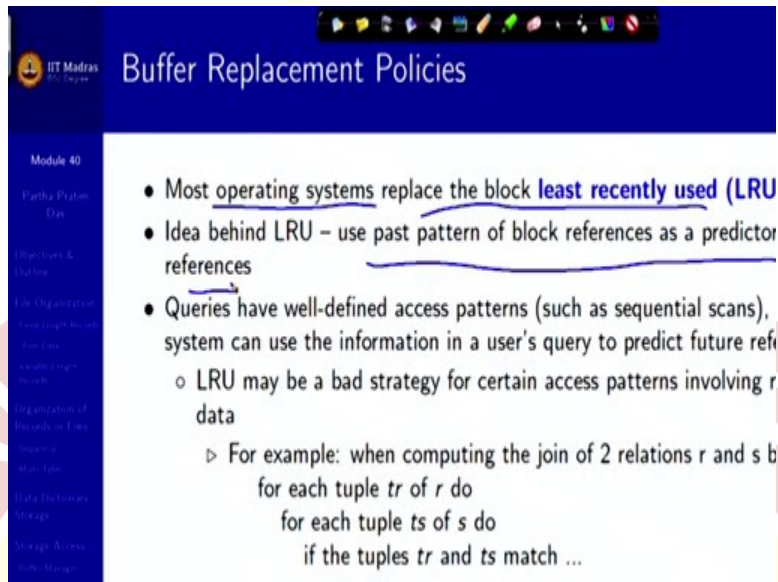
Buffer Manager

Module 40

- Programs call on the buffer manager when they need a block from disk
 - If the block is already in the buffer, buffer manager returns the address in main memory
 - If the block is not in the buffer, the buffer manager
 - ▷ Allocates space in the buffer for the block
 - Replacing (throwing out) some other block, if required, to the new block
 - Replaced block written back to disk only if it was modified since the recent time that it was written to / fetched from the disk
 - ▷ Reads the block from the disk to the buffer, and returns the address in main memory to requester

Once the allocation gets completed you read the block from the disk to the buffer and return the address, these are basic logical steps involved in the management of the buffer which sits between the program and the disk to make sure that you, the program will always get its block in the buffer.

(Refer Slide Time: 26:48)



Buffer Replacement Policies

- Most operating systems replace the block least recently used (LRU)
- Idea behind LRU – use past pattern of block references as a predictor references
- Queries have well-defined access patterns (such as sequential scans), system can use the information in a user's query to predict future references
 - LRU may be a bad strategy for certain access patterns involving r data
 - ▷ For example: when computing the join of 2 relations r and s
for each tuple tr of r do
for each tuple ts of s do
if the tuples tr and ts match ...

Now, certainly when you decide to replace the question is the buffer has multiple blocks which one would you replace? If you have studied operating system I am not sure if you have, then you have come across this issue of page replacement policies and so on, so it is basically very similar though the, I should say the consideration factors may be different in this case, but the strategies are similar.

So, which has one strong strategy known as LRU 'Least Recently Used'. What is least recently used is a block which in terms of use is the oldest, it may not be oldest in terms of fetch but in terms of use it is the oldest. So, if it has not been used for the longest period of time, then you expect that well the program is no more interested in it, so you can use that space. So, the past pattern of references is a predictor of the future reference, that is the basic idea.

(Refer Slide Time: 27:53)

Buffer Replacement Policies

- Most operating systems replace the block **least recently used (LRU)**
- Idea behind LRU – use past pattern of block references as a predictor references
- Queries have well-defined access patterns (such as sequential scans), system can use the information in a user's query to predict future references
 - LRU may be a bad strategy for certain access patterns involving r data
 - ▷ For example: when computing the join of 2 relations r and s b
for each tuple tr of r do
for each tuple ts of s do
if the tuples tr and ts match ...

Buffer Replacement Policies

- Most operating systems replace the block **least recently used (LRU)**
- Idea behind LRU – use past pattern of block references as a predictor references
- Queries have well-defined access patterns (such as sequential scans), system can use the information in a user's query to predict future references
 - LRU may be a bad strategy for certain access patterns involving r data
 - ▷ For example: when computing the join of 2 relations r and s b
for each tuple tr of r do
for each tuple ts of s do
if the tuples tr and ts match ...

Now, queries often have well defined access pattern, such as it may have a sequential scan. And the database system can use that information to predict future references, this is something which is, which often is different from the kind of challenge that an operating system has, where the predictions are a different ballgame, but here it often is follows a certain query structure.

And LRU may be a bad strategy for certain access patterns, for example, you are doing join, so you have nested loops. If you have nested loops, then you are going to repeatedly use in the inner, when you are working in the inner loop, you will, you are going to repeatedly use outer

loop stuff again in the next iteration. So, if that turns out to be your LRU which it might, then you are replacing and bringing back the same block multiple times.

So, I am just talking about giving you pointers to what could be the different considerations which might lead to a mixed strategy on the replacement policy that you are going to use and often that comes from the query optimizer.

(Refer Slide Time: 29:19)

Buffer Replacement Policies (2)

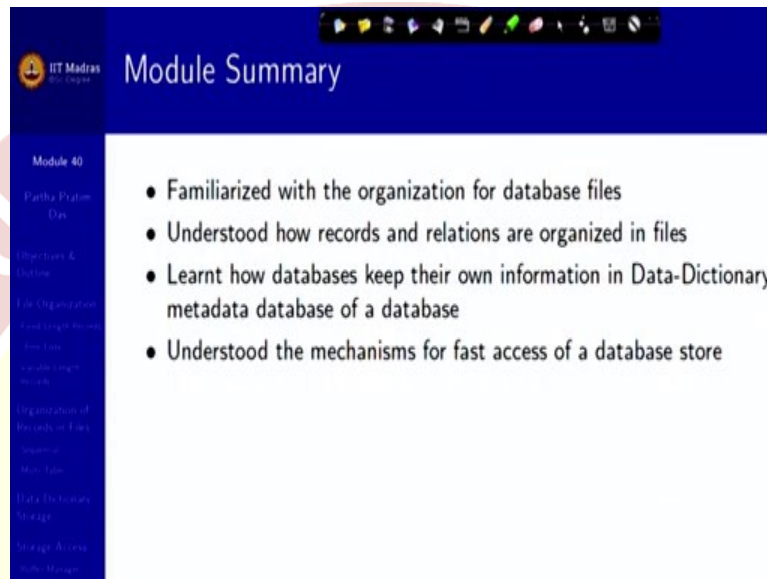
- **Pinned block:** memory block that is not allowed to be written back
- **Toss-immediate strategy:** frees the space occupied by a block as soon as the final tuple of that block has been processed
- **Most recently used (MRU) strategy:** system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinned and it becomes the most recently used block.
- Buffer manager can use statistical information regarding the probability that it will reference a particular relation
 - For example., the data dictionary is frequently accessed. Heuristic: keep data-dictionary blocks in main memory buffer
- Buffer managers also support **forced output** of blocks for the purpose of forcing a write back

So, some of the policies involved, one is pinned block, pinned block is one you can pin a block so say that this memory block cannot be written back. So, you can do a toss immediate strategy, where the space occupied by the block is considered free as soon as the block has been processed, it then have a most recently used strategy that is a system pins, that is you cannot replace the block currently being processed, and after the final tuple of the block has been processed it is unpinned, so it becomes a most recently used block.

So, buffer manager in general can use several statistical information regarding the probability and keep the data dictionary blocks mostly in the main memory, its own data dictionary that is a heuristic, as I was saying that, data dictionary you are going to use everytime. So, what is the point bringing it every time, you should have a separate allocated space in the buffer where you just keep it in the memory.

So, these are, it can also support forced output of blocks for the purpose of recovery that you write back because you need to do a recovery and so on. So, the different buffer replacement policies that is goes on to make sure that you have a better and better performance.

(Refer Slide Time: 30:50)



So, with that we come to the end of this module, we have given a fair idea about the organization of typical database files, how the records are laid out in that, what is the importance of data dictionary storage and some of the storage access mechanism particularly the buffers and buffer management strategies.

Thank you very much for your attention, going forward in the next week we will start talking about indexing and you will see the impact of these design considerations on indexing and how to make, what are the different ways to make accesses more and more fast.